

Optional

****Report your browser version, CPU type, cache size, RAM amount, and OS. We use this information to learn about the attack's behavior on different machines.****

- Browser: Zen (Firefox)
- CPU: intel core vpro i7
- Cache sizes: 64 Bytes
- RAM: 32GB
- OS: Ubuntu

1-2

****Use the values printed on the webpage to find the median access time and report your results as follows.****

Number of Cache Lines	Median Access Latency (ms)	
1	0	
10	0	
100	0	
1,000	0	Start to see 1's show
10,000	0	
100,000	1	
1,000,000	8	
10,000,000	82	

1-3

****According to your measurement results, what is the resolution of your `performance.now()`? In order to measure differences in time with `performance.now()`, approximately how many cache accesses need to be performed?****

The resolution of `performance.now()` is 1 ms, which lines up with their experiments.

The first non-zero median appears at $N = 100,000$ cache lines. There were a few times a 1 would show at 1,000 and 10,000. Therefore, approximately 100,000 cache accesses must be performed in a single timed region in order to produce a measurable, non-zero reading.

2-2

****Report important parameters used in your attack. For each sweep operation, you access N addresses, and you count the number of sweep operations within**

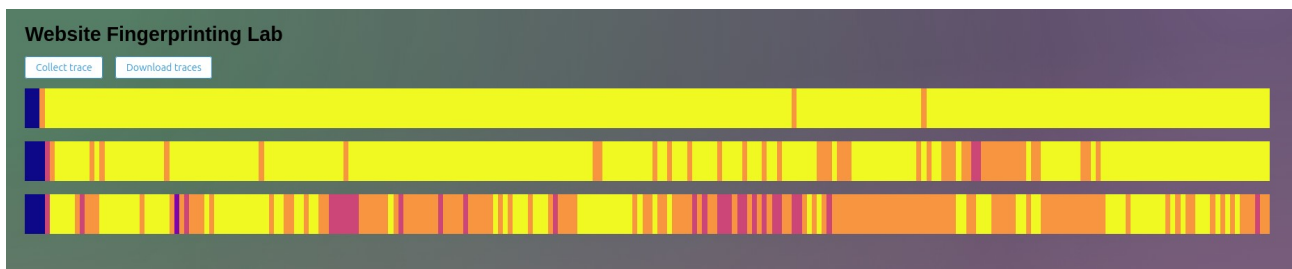
a time interval P ms. What values of N and P do you use? How do you choose N ? Why do not you choose P to be larger or smaller?*

$N = 200,000$, $P = 20$ ms

N is chosen so that the array is large enough to push out any data the target page loaded into the cache, making the page load detectable. $P = 20$ ms worked well – a larger P misses the detail of the page load, and a smaller P doesn't give enough sweeps per slot to get a clear reading.

2-3

Take screenshots of the three traces generated by your attack code and include them in the lab report.



2-4

Use the Python code we provided in Part 2.1 to analyze simple statistics (mean, median, etc.) on the traces from google.com and nytimes.com. Report the statistic numbers.

Mean = 25.1435
Median = 24.0
Max = 37
Min = 5

2-6

Include your classification results in your report.

...

	precision	recall	f1-score	support
https://www.antiochorlando.com	0.96	1.00	0.98	47
https://www.ign.com	0.75	0.77	0.76	31
https://www.planningcenter.com	0.92	0.80	0.86	41
https://www.youtube.com	0.95	1.00	0.98	41
accuracy			0.91	160
macro avg	0.89	0.89	0.89	160
weighted avg	0.91	0.91	0.91	160

...

3-2

****Include your new accuracy results for the modified attack code in your report.****

...

	precision	recall	f1-score	support
https://www.antiochorlando.com	0.86	0.91	0.89	47
https://www.ign.com	0.93	0.90	0.92	31
https://www.planningcenter.com	1.00	0.98	0.99	41
https://www.youtube.com	0.93	0.90	0.91	41
accuracy			0.93	160
macro avg	0.93	0.92	0.93	160
weighted avg	0.93	0.93	0.93	160

...

3-3

****Compare your accuracy numbers between Part 2 and 3. Does the accuracy decrease in Part 3? Do you think that our “cache-occupancy” attack actually exploits a cache side channel? If not, take a guess as to possible root causes of the modified attack.****

From this one test, the final accuracy in Part 3 is slightly higher than in Part 2. This suggests that the modified attack still captures some distinguishable patterns in the traces. It is possible that the attack is not purely exploiting a cache side channel but rather other timing differences or artifacts in the JavaScript execution environment. Something else I've noticed is how some of the scores are both higher and lower in Part 3 compared to part 2, which suggests that the attack may be capturing some different features of the page loads.